EJ1 – Identificación de paradigmas de desarrollo de IA

Solución Conceptual -> Tabla de Asociación Percepción-Acción:

La función de decisión (inferencia) puede ser un simple esquema de asociación (una tabla) o un autómata finito. La lógica de las "percepciones" se especifica a un nivel muy bajo para que el tiempo de cálculo sea muy pequeño y permita al agente operar de forma reactiva.

Ejercicio 3: Paradigma Situado - Agente de Navegación Reactiva

Diseñar la lógica de decisión de un **Agente Vehículo Autónomo de Almacén (AGV)**. El AGV debe ejecutar una sola conducta elemental: Evitar Obstáculos Inmediatamente para avanzar hacia su destino (no se requiere planificación a largo plazo, solo respuesta reactiva)

El AGV tiene tres sensores de proximidad (Sensores = S). La lectura es binaria, ya que está especificada a un nivel muy bajo para garantizar un tiempo de cálculo pequeño y la operación en tiempo real:

Sensor (Percepción)	Posición	Valor
S1	Adelante	1 = Obstáculo Detectado / 0 = Vía Libre
S2	Izquierda	1 = Obstáculo Detectado / 0 = Vía Libre
S3	Derecha	1 = Obstáculo Detectado / 0 = Vía Libre

Acciones Disponibles (Efectores/Conductas):

- **A:** Avanzar (Conducta por defecto, si no hay peligro).
- **D:** Detener (Acción de emergencia).
- **GI:** Girar 90° Izquierda.
- **GD:** Girar 90° Derecha.

Posible respuesta:

Sensor Adelante (S1)	Sensor Izquierda (S2)	Sensor Derecha (S3)	Acción Reactiva (Conducta)
0	0	0	Avanzar
1	0	0	Detener y Girar a la Izquierda / Derecha
0	1	0	Girar a la Derecha
0	0	1	Girar a la Izquierda
1	1	0	Detenerse y Girar a la Derecha
0	1	1	Avanzar
1	0	1	Detenerse y girar a la Izquierda
1	1	1	Detenerse o hacer un doble giro

Tarea 2:

Programa el algoritmo creando las reglas de percepción-acción en un diccionario.

Como no tenemos forma de saber cuándo llega el AGV a su destino programaremos una simulación basada en:

- La creación de un diccionario donde la clave sea una tupla con el estado de los sensores (S1, S2, S3) y el valor sea la acción correspondiente que sirva como tabla de percepción-acción
- Simula el recorrido del AGV con una lista de tuplas que representen las percepciones (mínimo 10)
- Recorre las percepciones y muestra la acción correspondiente según tu tabla.
- Si hay una percepción que no está en la tabla muestra: Detener.

```
tabla_percepcion_accion = {
    (0, 0, 0): "Avanzar",                                # Todo despejado
    (1, 0, 0): "Detener y Girar a la Derecha",         # Obstáculo
adelante
    (0, 1, 0): "Girar a la Derecha",                   # Obstáculo
izquierda
    (0, 0, 1): "Girar a la Izquierda",                 # Obstáculo derecha
    (1, 1, 0): "Detener y Girar a la Derecha",         # Obstáculos
adelante e izquierda
    (0, 1, 1): "Avanzar",                                # Solo laterales
bloqueados
    (1, 0, 1): "Detener y Girar a la Izquierda",       # Obstáculos
adelante y derecha
    (1, 1, 1): "Detener"                                 # Todos los
sensores bloqueados
}
#- Simula el recorrido del AGV con una lista de tuplas que representen
las percepciones (mínimo 10).
# Si hay una percepcion que no esta en la tabla, muestra: Detener.
percepciones = [
    (0, 0, 0),  # Todo despejado
    (1, 0, 0),  # Obstáculo adelante
    (0, 1, 0),  # Obstáculo izquierda
    (0, 0, 1),  # Obstáculo derecha
    (1, 1, 0),  # Obstáculos adelante e izquierda
    (0, 1, 1),  # Solo laterales bloqueados
    (1, 0, 1),  # Obstáculos adelante y derecha
    (1, 1, 1),  # Todos los sensores bloqueados
    (0, 0, 0),  # Todo despejado nuevamente
    (1, 0, 1),   # Obstáculos adelante y derecha
    # ejemplo de percepcion que no esta en la tabla: detener
    (1, 1, 1)   # Todos los sensores bloqueados
]

# Para cada percepcion del recorrido, mostrar la accion correspondiente
for percepcion in percepciones:
    accion = tabla_percepcion_accion.get(percepcion, "Detener")
    print(f"Percepción: {percepcion} -> Acción: {accion}")
```